CSDS 340 Case Study: Apple Quality Classification

Group 18

Spring 2026

1 Goal

The case study asks for a complete binary classification pipeline on the Apple Quality dataset using scikit-learn: we compare logistic regression, support vector machines, and decision trees, apply appropriate data preprocessing, and use hyperparameter tuning to improve predictive accuracy.

2 Data preprocessing and its impact

Source and shape: We load `train.csv` with pandas and convert to NumPy. The first seven columns are continuous features (Size, Weight, Sweetness, Crunchiness, Juiciness, Ripeness, Acidity); the eighth column is the binary label `Quality` (0 bad, 1 good). The training file contains 3,200 rows (excluding the header). Inspection of the raw values shows no missing entries, so no imputation was required.

Feature scale: The raw feature values are already approximately centred near zero with moderate spread (roughly -5 to $+6$), suggesting the data may have been pre-standardised upstream. `StandardScaler` is nevertheless applied as the first pipeline step: it is fit exclusively on the training portion of each cross-validation fold and then applied to the held-out fold, so no fold sees statistics derived from its own data. In practice this step has minimal numerical effect given the existing scale, but it guarantees correctness and keeps all three pipelines consistent.

Impact of scaling by model: Scaling is critical for logistic regression and SVM: both are sensitive to feature magnitudes (gradient descent and margin maximisation respectively assume comparable scales). Decision trees are invariant to monotone feature transforms, axis-aligned splits are unaffected by rescaling, so `StandardScaler` has no impact on DT results, but is retained for pipeline uniformity.

Class balance: Label counts are $(1584, 1616)$ for the two classes, nearly balanced. Accuracy is therefore a reasonable primary metric and no resampling or class weighting was applied.

Dimensionality reduction: We do not apply PCA or LDA as a default preprocessing step: with only seven features, reducing further risks discarding discriminative signal. A targeted experiment adding PCA before the decision tree (Section 3.3) confirmed this, all seven components were selected, meaning no compression helped.

Validation strategy. All model selection uses 5-fold stratified cross-validation (`StratifiedKFold`, `shuffle=True`, `random_state=42`), preserving the class ratio in every fold. Some logistic regression runs used $k = 10$ for comparison; results were consistent with $k = 5$.

3 Models and parameters

Each family is implemented with the corresponding scikit-learn estimator; hyperparameters are searched with `GridSearchCV` and `scoring='accuracy'` unless noted otherwise.

Tuning process and why it works: `GridSearchCV` exhaustively evaluates every combination of hyperparameter values in a specified grid. For each combination, the estimator is trained and scored using k-fold cross-validation rather than a single train/test split: the training data is divided into k folds, the model is fit on $k-1$ folds and evaluated on the held-out fold, and this is repeated k times so every sample is used for validation exactly once. The final score is the mean accuracy across all k folds, which gives a lower-variance estimate of generalisation performance than any single split would. Stratified folds (`StratifiedKFold`) are used so the class ratio is preserved in every fold, preventing any fold from being accidentally dominated by one class.

Crucially, each model is wrapped in a `Pipeline` that places `StandardScaler` before the classifier. Inside `GridSearchCV`, the entire pipeline is re-fit from scratch on each training fold, meaning the scaler computes its mean and variance only from the training portion of that fold and never sees the held-out data. This prevents data leakage: if the scaler were fit on the full dataset first, the validation fold would carry information from its own statistics into the model, inflating the reported accuracy. By searching over the full grid with this leakage-free evaluation, `GridSearchCV` identifies the hyperparameter combination that is expected to generalise best to unseen data.

3.1 Logistic regression

Parameters varied:

- **Solver:** `lbfgs`, `liblinear`, `newton-cg`, `newton-cholesky`, `sag`, `saga` (each paired only with penalties and options that solver supports).
- **Inverse regularization C:** $\{0.01, 0.1, 1, 10, 100\}$ where applicable.
- **SAGA penalties:** grids for `elasticnet` (with `l1_ratio` $\in \{0, 0.5, 1\}$), `l1`/`l2`, and `penalty=None`.

The search uses a *list* of parameter dictionaries in `GridSearchCV` so invalid combinations (e.g. `l1_ratio` without elastic net) are never evaluated.

3.2 Support vector machine

Parameters varied:

- **Kernel:** `rbf`, `linear`, `poly` — controls the shape of the decision boundary.
- **C (regularisation):** trades off margin width against training error; higher C fits the training data more tightly.
- **γ (RBF width):** controls how far the influence of a single training example reaches; smaller values give smoother boundaries.

- **Polynomial `degree`:** for the poly kernel only, sets the polynomial order.
- **`nu` (NuSVC):** alternative parameterisation bounding the fraction of margin errors, explored as a complement to C.

Runs 1 to 4 - kernel and C selection: An initial coarse search over RBF and linear kernels with $C \in \{0.1, 1, 10, 100\}$ identified RBF as the best kernel. C was then refined over $\{10, 20, \ldots, 100\}$, then tightened to $\{60, 65, 70, 75\}$. Best: `kernel=rbf`, $C = 70$, `gamma=scale`, CV accuracy **0.9094**.

Run 5 - polynomial kernel: $C \in \{10, 50, 100\}$, `degree` $\in \{2, 3\}$, `gamma` $\in \{\texttt{scale}, \texttt{auto}\}$. Best: $C = 100$, `degree=3`, `gamma=scale`, CV accuracy **0.8244**. Worse than RBF.

Run 6 - hand-tuned numeric gamma: $C \in \{50, 60, 70, 80, 90, 100\}$, `gamma` $\in \{0.001, 0.01, 0.1, 0.5, 1, \texttt{scale}\}$. `scale` still performed best, no gain.

Run 7 - polynomial feature expansion before SVM: `PolynomialFeatures(degree=2)` inserted before the scaler; RBF kernel with $C \in \{0.1, 1, 10, 70, 100, 1000\}$. Best: $C = 10$, `gamma=scale`, CV accuracy **0.9062**. Slight regression; feature expansion dropped.

Run 8 - fine gamma neighbourhood + NuSVC: `gamma=scale` computes to $1/(n_{\text{features}} \times \text{var}) = 1/7 \approx 0.143$ after `StandardScaler`. A tight grid $\{0.08, 0.10, 0.12, 0.14, 0.16, 0.18, 0.20\}$ around that value was searched with $C = 70$ fixed. `NuSVC` with `nu` $\in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ was also evaluated as an alternative parameterisation. Best SVC: `gamma=0.16`, CV accuracy **0.9109** (+0.0015 over run 4). NuSVC best: `nu=0.2`, CV accuracy 0.9091; no gain over SVC.

Run 9 - confirm gamma peak: Tighter grid $\{0.14, 0.15, 0.16, 0.17, 0.18\}$ confirmed `gamma=0.16` is the true optimum; neighbouring values 0.15 and 0.17 both scored lower.

3.3 Decision tree

Parameters varied:

- **`criterion`:** split quality measure (`gini` or `entropy`).
- **`max_depth`:** maximum tree depth; limits model complexity and controls overfitting.
- **`min_samples_leaf`:** minimum samples required at a leaf node; higher values smooth the model.
- **`min_samples_split`:** minimum samples required to split an internal node.
- **`max_features`:** number of features considered at each split (`None`, `sqrt`, `log2`).
- **`ccp_alpha` (cost-complexity pruning):** penalises tree complexity post-hoc; higher values yield smaller trees.
- **PCA `n_components`:** number of principal components retained before the classifier.

Run 1 - structural hyperparameters: A joint grid search over five structural parameters was performed/ Best configuration: `criterion=gini`, `max_depth=12`, `max_features=None`, `min_samples_leaf=2`, `min_samples_split=10`. CV accuracy: **0.8134**.

Run 2 - cost-complexity pruning (`ccp_alpha`): The structural parameters were fixed to the run 1 optimum and post-pruning was explored via `ccp_alpha`. Candidate α values were derived from the pruning path of a single tree fit on the full scaled training set using `cost_complexity_pruning_path`, then sampled at 20 log-spaced points across the path (plus $\alpha = 0$). . Best configuration: `ccp_alpha` ≈ 0.00101. CV accuracy: **0.8184**

Run 3 - PCA before decision tree: All best parameters from runs 1 and 2 were fixed and `PCA` was inserted between the scaler and the classifier, with `n_components` $\in \{1, 2, 3, 4, 5, 6, 7\}$ as the only free variable. Best: `n_components=7` (all features retained). CV accuracy **0.8184**, identical to run 2. PCA provided no benefit.

4 Final choice and reasoning

Chosen model: We select `SVC(kernel='rbf', C=70, gamma=0.16)` preceded by `StandardScaler` as the final classification model. This configuration achieved the highest 5-fold stratified CV accuracy of **91.09%**, outperforming logistic regression by ∼17 percentage points and the decision tree by ∼9 percentage points.

Why SVM over the alternatives. Logistic regression is limited to linear decision boundaries; despite exhaustive tuning across all six solvers and every supported penalty scheme, accuracy plateaued at 74.25%, confirming the apple quality boundary is not linearly separable. The decision tree can capture non-linearity through axis-aligned splits, but only in a piecewise, high-variance manner: structural tuning and cost-complexity pruning pushed it to 81.84%, with PCA confirming the ceiling is structural rather than a feature representation issue. The RBF kernel implicitly maps data to an infinite-dimensional space and learns a smooth, global non-linear boundary, making it structurally better suited to this problem than either alternative.

Hyperparameter tuning strategy. C and γ were tuned in two stages: a coarse grid search first identified $C = 70$ and `gamma=scale` as strong candidates, then a fine-grained search around the computed scale value ($1/7 \approx 0.143$) revealed $\gamma = 0.16$ as the true optimum. This two-stage approach avoids the cost of an exhaustive fine grid while still finding the true peak, as confirmed by the tighter run 9 grid where neighbouring values 0.15 and 0.17 both scored lower.

Learning curve and train/CV gap. Figure 1 shows the learning curve for the chosen model. Training accuracy stabilises around 96% while CV accuracy converges to ∼91%, leaving a persistent gap of approximately 5 percentage points. This gap indicates mild overfitting: the model fits the training data slightly more tightly than it generalises across held-out folds. The cause is the high value of $C = 70$, which allows few margin errors and therefore a tighter fit to training examples. Importantly, the CV curve has fully converged by ∼2500 samples and the standard deviation across folds is narrow at the right end of the curve, meaning the 91% estimate is stable and not an artefact of a particular fold split. The gap is an accepted trade-off: lowering C to reduce it was explored during tuning and consistently hurt CV accuracy, so the chosen $C = 70$ represents the best achievable balance between fit and generalisation on this dataset.

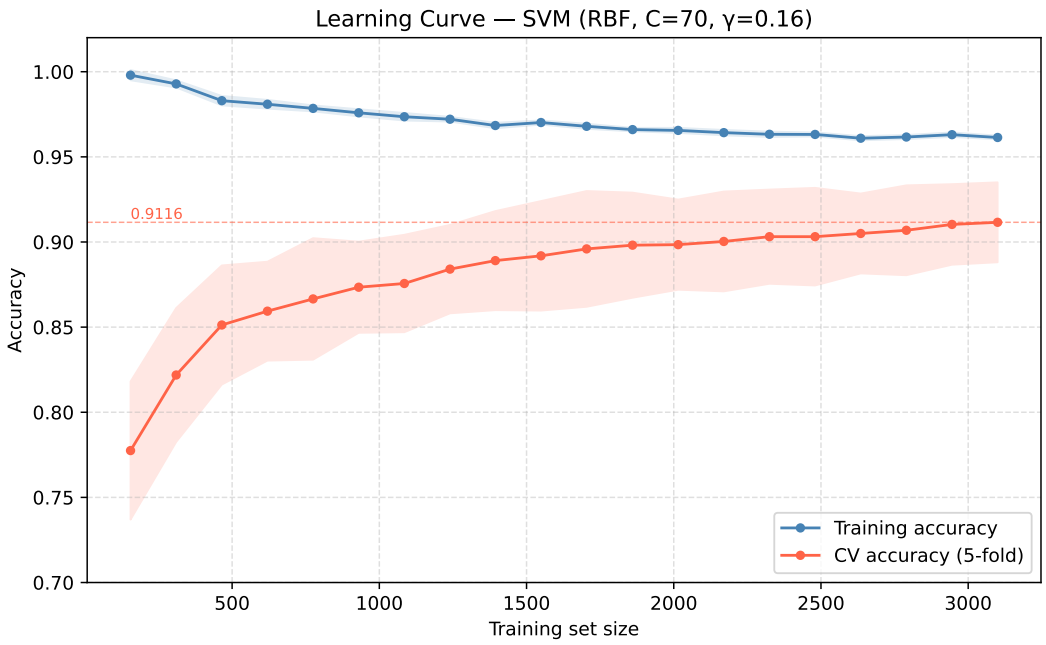

Figure 1: Learning curve for `SVC(kernel=rbf, C=70, γ=0.16)`. Shaded bands show ±1 standard deviation across folds.

5 Conclusion

Table 1 summarises the best cross-validation accuracy achieved by each model after full hyperparameter tuning.

Model	Best hyperparameters	CV accuracy
Logistic Regression	`saga`, `elasticnet`, $C = 0.1$, `l1_ratio=1`	0.7425
Decision Tree	`gini`, `max_depth=12`, `ccp_alpha`≈ 0.00101	0.8184
SVM	`rbf`, $C = 70$, $\gamma = 0.16$	**0.9109**

Table 1: Best 5-fold stratified CV accuracy for each model family.

The SVM with an RBF kernel is the clear winner, outperforming logistic regression by ∼17 percentage points and the decision tree by ∼9 percentage points. The magnitude of these gaps is informative: the large margin over logistic regression confirms the apple quality boundary is genuinely non-linear; the gap over the decision tree shows that smooth non-linear structure (captured well by the RBF kernel) is harder to approximate with axis-aligned splits.

Selected model for test set evaluation: `SVC(kernel='rbf', C=70, gamma=0.16)`, preceded by `StandardScaler`.